

Sistema de Reconciliación de Pagos Multi-Fuente

Documentación técnica completa
del código, carpeta por carpeta y archivo por archivo

Emanuel Villada Muñetón
Prueba técnica de automatización
Python 3.12 · 61 archivos · 172 pruebas

Índice

1. El problema que resuelve

2. Cómo está organizado el proyecto

3. El recorrido completo de una ejecución

4. Archivos de la raíz

5. reconciliacion/ — infraestructura transversal

6. config/ — configuración de rutas

7. dominio/ — el vocabulario del negocio

8. loaders/ — leer las tres fuentes
9. limpieza/ — interpretar lo sucio

10. procesamiento/ — decidir

11. exportadores/ — presentar

12. gui/ — la interfaz

13. tests/ — las pruebas

14. Carpetas de datos y salidas

15. Patrones de diseño aplicados

16. Reglas de negocio, de referencia

1. El problema que resuelve

Tres sistemas distintos describen la misma transacción de pago desde ángulos diferentes, y no siempre coinciden. El objetivo es cruzarlos, encontrar dónde dejan de coincidir, detectar patrones de fraude y entregar todo en un Excel que un área contable pueda usar.

Fuente	Archivo	Qué representa	Cómo llama a la llave	Registros
CSV	autorizaciones.csv	Lo que se autorizó	ID_Transaccion	500
SQLite	reconciliacion_pagos.db	Lo que se contabilizó	Referencia	480
JSON	movimientos_bancarios.json	Lo que llegó al banco	transaccion_id	465

El universo a analizar es la **unión** de los tres: **505 transacciones**. Toda transacción que aparezca en al menos una fuente debe salir en el reporte y quedar clasificada.

La dificultad real no es el cruce, es el CSV. Sus campos Monto y Marca no son un número y un texto: son estructuras tipo JSON deliberadamente rotas, con seis variantes de corrupción conviviendo en el mismo archivo. Un `json.loads()` falla en las 500 filas.

2. Cómo está organizado el proyecto

reconciliacion-pagos-multifuentes/		
├─ datos/		Las tres fuentes de entrada
├─ docs/		Esta documentación y el manual de uso
├─ capturas/		Imágenes de la interfaz para el README
├─ salida/		Donde queda el Excel generado
├─ reconciliacion/		El paquete con toda la lógica
│ ├─ config/rutas.py		Todas las rutas, en un solo lugar
│ ├─ dominio/		Modelos, enumeraciones, la transacción cruzada
│ ├─ loaders/		Un cargador por fuente, con contrato común
│ ├─ limpieza/		Extracción de los campos malformados
│ ├─ procesamiento/		Reconciliación, reglas y detección de fraude
│ ├─ exportadores/		Generación del Excel
│ ├─ gui/		Interfaz Tkinter + customtkinter
│ ├─ errores.py		Excepciones propias
│ ├─ log.py		Configuración de logs
│ ├─ progreso.py		Reporte de avance parcial
│ └─ servicio.py		Orquestador de todo el proceso
├─ tests/		172 pruebas con pytest
├─ main.py		Ejecutar por terminal
├─ ejecutar_gui.py / .bat		Ejecutar con interfaz / con doble clic
└─ README.md		Decisiones de diseño

El principio que ordena todo

Cada capa tiene **una sola razón para cambiar**:

Capa	Su única responsabilidad	Lo que NO hace
loaders	Leer bytes del disco y convertirlos a objetos	No interpreta campos sucios
limpieza	Interpretar texto malformado	No toca disco
procesamiento	Decidir qué significa cada cruce	No sabe de archivos ni de Excel
exportadores	Presentar el resultado	No calcula nada
gui	Mostrar y recibir clics	No contiene lógica de negocio

La consecuencia práctica: el parseo de un campo corrupto se prueba pasándole un *string*, sin archivos de por medio; y agregar una cuarta fuente mañana solo obliga a escribir un cargador nuevo.

3. El recorrido completo de una ejecución

Esto es lo que pasa, en orden, desde que se pulsa el botón hasta que aparece el Excel:

#	Paso	Quién lo hace	Resultado
1	Verificar que existan los tres archivos	<code>servicio.py</code> → <code>rutas.py</code>	Si falta uno, se detiene con un mensaje legible
2	Leer el CSV	<code>loaders/csv_loader.py</code>	500 filas crudas , sin interpretar
3	Leer SQLite	<code>loaders/sqlite_loader.py</code>	480 contabilizaciones
4	Leer el JSON	<code>loaders/json_loader.py</code>	465 movimientos
5	Limpiar el CSV	<code>limpieza/autorizaciones.py</code>	500 autorizaciones con monto, marca y retenciones
6	Cruzar por el identificador	<code>procesamiento/reconciliador.py</code>	505 transacciones (la unión)
7	Clasificar cada transacción	<code>procesamiento/reglas.py</code>	Etiquetas y observaciones
8	Buscar patrones de fraude	<code>procesamiento/fraude.py</code>	149 marcadas, con nivel de riesgo
9	Escribir el Excel	<code>exportadores/excel.py</code>	505 filas × 29 columnas, con colores

Todo el proceso tarda alrededor de **1,2 segundos**. Cada paso informa su avance, y ese aviso es el que mueve la barra de la interfaz.

4. Archivos de la raíz

`main.py`

98 líneas

Qué es: el script ejecutable que corre todo el proceso por terminal y genera el Excel.

Qué contiene: `main()` como punto de entrada, `_mostrar_progreso()` que imprime cada avance, y `_mostrar_resumen()` que al final imprime los indicadores.

Decisión: no contiene ni una línea de lógica de negocio. Solo llama a `ServicioReconciliacion` y da formato a la salida. Devuelve código `0` si todo salió bien y `1` si falló por algo previsible, que es lo que espera cualquier automatización que lo invoque.

`ejecutar_gui.py`

15 líneas

Qué es: el punto de entrada de la interfaz gráfica. Importa `lanzar()` y la ejecuta. Deliberadamente mínimo.

Qué es: el archivo que permite abrir la aplicación con doble clic, sin terminal. Es el punto extra que pedía el enunciado.

Qué hace: se sitúa en su propia carpeta, prefiere el entorno virtual del proyecto si existe, comprueba que Python y las dependencias estén, y lanza la aplicación con `pythonw` (que no abre consola).

Por qué comprueba antes de lanzar: `pythonw` no muestra errores. Si faltara una dependencia, el doble clic no haría absolutamente nada y la persona no sabría por qué. Con la comprobación previa, aparece un mensaje que explica qué instalar.

Ejecución: `openpyxl` (Excel), `customtkinter` (interfaz), `pytest` (pruebas).

Desarrollo: agrega `pytest-cov` (cobertura), `mypy` y `types-openpyxl` (verificación de tipos). Separarlos evita que quien solo quiera correr la app instale herramientas que no necesita.

pytest.ini: dice dónde están las pruebas y cómo se llaman.

mypy.ini: activa el **modo estricto** de verificación de tipos. Las dos únicas excepciones son para `openpyxl` y `customtkinter`, que no publican anotaciones; están acotadas al módulo que las usa, no desactivadas globalmente.

Qué es: integración continua. En cada cambio subido a GitHub instala las dependencias, corre las 172 pruebas con cobertura y verifica los tipos. Si algo se rompe, se ve en el repositorio y no en la máquina de quien lo clone.

5. reconciliacion/ — infraestructura transversal

Cuatro archivos que no pertenecen a ninguna capa concreta porque las atraviesan todas.

reconciliacion/servicio.py

216 líneas

Qué es: el orquestador. El único punto que conoce el proceso de principio a fin.

Qué contiene:

- `ServicioReconciliacion.ejecutar(progreso, exportar)` — corre los nueve pasos en orden.
- `ResultadoProceso` — todo lo que produce una ejecución: transacciones, resúmenes, advertencias, ruta del reporte y duración.
- `_parcial()` — traduce el avance de una etapa (por ejemplo «fila 320 de 505») al porcentaje global del proceso.
- `_verificar_fuentes()` — comprueba los tres archivos de una vez.

La decisión más importante del proyecto: este servicio existe para que la terminal y la interfaz ejecuten **exactamente el mismo código**. La GUI no reimplementa nada; solo le pasa una función de progreso distinta. Es lo que evita que uno de los dos quede desactualizado.

Por qué verifica las tres fuentes juntas: para poder decirle al usuario *todo* lo que le falta en un solo mensaje, en vez de hacerlo fallar tres veces seguidas.

reconciliacion/errores.py

45 líneas

Qué es: la jerarquía de excepciones propias.

Excepción	Cuándo se lanza
<code>ErrorReconciliacion</code>	Base. Lleva un <code>mensaje</code> apto para el usuario y un <code>detalle</code> técnico para el log
<code>FuenteNoDisponibleError</code>	No se encontró un archivo de datos
<code>FuenteCorruptaError</code>	El archivo existe pero no se puede interpretar
<code>ErrorExportacion</code>	No se pudo escribir el Excel (por ejemplo, está abierto)
<code>ProcesoCancelado</code>	El usuario detuvo el proceso. No es un fallo

Por qué una jerarquía propia: para que la interfaz distinga un problema *previsible* del negocio de un fallo inesperado del programa, y traduzca el primero a un mensaje en el idioma del usuario en vez de mostrar un *traceback*.

Qué es: la configuración del sistema de logs.

Qué contiene: `obtener_logger()`, que devuelve el logger del módulo que lo pide, y `configurar_consola()`, que solo usa la versión de terminal.

Regla que se cumple en todo el proyecto: el código de negocio **nunca imprime**, solo registra. Por eso el mismo código sirve sin cambios para la terminal y para la interfaz: la GUI engancha su propio manejador y recibe los mismos mensajes mientras el proceso corre. No hay ni un `print` en todo el paquete.

También fuerza la salida a UTF-8 tolerante, porque la consola de Windows suele venir en cp1252 y fallaría al escribir «CRÍTICO» con tilde.

Qué es: el mecanismo para informar avance *dentro* de las etapas largas.

Qué contiene: el tipo `ProgresoParcial` —una función que recibe (procesados, total)— y `notificar()`, que avisa cada 20 elementos y siempre en el último.

Por qué existe: limpiar 500 filas o escribir 505 filas de Excel toma lo suficiente como para que una interfaz que solo se entera al terminar cada etapa parezca congelada. El paquete de negocio no sabe nada de hilos ni de ventanas: solo avisa, y quien lo llama decide qué hacer con el aviso.

6. `config/` — configuración de rutas

`reconciliacion/config/rutas.py`

57 líneas

Qué es: la única fuente de verdad sobre dónde viven los archivos.

Qué contiene: las rutas de las tres fuentes y del reporte, `FUENTES_REQUERIDAS` (nombre legible → ruta), `asegurar_directorio_salida()` y `fuentes_faltantes()`.

Decisiones:

- Las rutas se resuelven **relativas a la raíz del repositorio**, así que el proyecto funciona sin cambios en cualquier máquina o carpeta.
- `fuentes_faltantes()` devuelve **nombres legibles** («Autorizaciones (CSV)») y no rutas crudas, porque ese texto va directo a la pantalla de una persona que no programa.

Es lo que permite cumplir el requisito de **no tener selectores de archivo**: la ruta es configuración del sistema, no una decisión que el usuario deba tomar cada vez.

7. `dominio/` — el vocabulario del negocio

`reconciliacion/dominio/modelos.py`

175 líneas

Qué es: una clase por concepto de negocio, en vez de diccionarios genéricos.

Clase	Qué representa	Origen
<code>FilaAutorizacionCruda</code>	Una fila del CSV tal cual está en disco, sin interpretar	CSV
<code>Autorizacion</code>	Lo que se autorizó, ya limpio	CSV
<code>Contabilizacion</code>	Lo que se contabilizó	SQLite
<code>MovimientoBancario</code>	Lo que llegó al banco	JSON
<code>Retencion</code>	Una retención (entidad + monto)	CSV

También define el vocabulario de estados: `AUTORIZADO`, `CONTABILIZADO` y `COMPLETADO` son el mismo estado «OK»; y los catálogos válidos por fuente, que se usan para validar integridad.

Por qué existe `FilaAutorizacionCruda` además de `Autorizacion`: para separar *leer* de *limpiar*. El cargador nunca interpreta y el parser nunca toca el disco. Gracias a eso, todo el parseo del JSON roto se prueba con cadenas de texto.

`Autorizacion.total_retencion(*entidades)` suma las retenciones pedidas: una entidad ausente aporta 0 y una repetida suma todas sus ocurrencias.

Qué es: el vocabulario cerrado que sale en el reporte.

- `Clasificacion` — `RECONCILIADO`, `DISCREPANCIA_MONTO`, `DISCREPANCIA_ESTADO`, `NO_ENCONTRADO_EN_JSON`, `NO_CONTABILIZADO`, `NO_AUTORIZADO`.
- `TipoFraude` — monto, hora, patrón, sin autorización.
- `NivelRiesgo` — crítico, alto, medio, con su tabla de prioridad.

Por qué enumeraciones y no cadenas sueltas: un error de escritura falla al importar y no silenciosamente en el Excel, y el valor exacto que se exporta vive en un solo lugar.

Qué es: el corazón del modelo. `TransaccionReconciliada` reúne las tres versiones de la misma transacción y expone, ya calculadas, las magnitudes que el resto del sistema necesita.

Propiedad	Qué calcula
<code>presente_csv / sqlite / json</code>	En qué fuentes aparece
<code>monto_csv / sqlite / json</code>	El monto según cada fuente
<code>monto_referencia</code>	El valor que coincide en la mayoría; si hay empate, el mayor
<code>diferencia_monto</code>	Máximo menos mínimo entre las fuentes presentes
<code>fecha_referencia</code>	La primera disponible: CSV → SQLite → JSON
<code>banco</code>	El de la primera fuente que lo reporte
<code>es_fraude / nivel_riesgo</code>	Si hay patrones y cuál es el más grave
<code>esta_reconciliada</code>	Si quedó sin ningún hallazgo

Criterio de reparto: las reglas que dependen *solo* de la transacción viven aquí como propiedades; las que dependen del conjunto —fraude por monto anómalo o por patrón— viven en el detector, porque necesitan mirar todas las transacciones a la vez.

Por qué el empate de monto se resuelve al mayor: criterio conservador. Es preferible sobreestimar la exposición que subestimarla.

Por qué la fecha de referencia prioriza el CSV: la autorización es el momento en que la operación realmente ocurre; la contabilización y el movimiento bancario son ecos posteriores de ese hecho.

8. loaders/ — leer las tres fuentes

reconciliacion/loaders/base.py

182 líneas

Qué es: el contrato común de los tres cargadores, con el patrón **Template Method**. `cargar()` fija los pasos y cada fuente solo implementa `_leer()`:

1. verificar que el archivo existe (común, aquí)
2. leer los registros (específico, en cada subclase)
3. validar la integridad de lo leído (común, aquí)
4. devolver un `ResultadoCarga` uniforme (común, aquí)

`ResultadoCarga` lleva los registros, las advertencias y —clave— `registros_leidos` aparte de los válidos, con la propiedad `hubo_perdida`.

Por qué se cuentan los leídos y los válidos por separado: para poder demostrar que **no se perdió ninguna fila en silencio**, que es lo que el enunciado exige verificar. Un proceso contable que descarta registros sin decirlo es peor que uno que falla.

Las validaciones comunes: identificador presente, identificador único dentro de la fuente, y `_validar_vocabulario()`, que comprueba que un campo solo traiga valores del catálogo esperado.

reconciliacion/loaders/csv_loader.py

122 líneas

Qué es: lee `autorizaciones.csv` (separado por `;`) y devuelve las filas **tal cual**, sin interpretar los campos malformados.

Valida que estén las seis columnas requeridas, descarta filas sin identificador dejando constancia del número de línea, y conserva `numero_fila` en cada registro para poder trazar cualquier problema hasta su origen exacto en el archivo.

reconciliacion/loaders/sqlite_loader.py

114 líneas

Qué es: lee la tabla `Contabilizaciones`. Es la única fuente con los campos ya limpios, la única que aporta `Centro_Costo` y la única cuyo estado puede ser `PENDIENTE` o `RECHAZADO`.

Se abre en modo solo lectura (`mode=ro`). Es una fuente de datos, no un almacén del sistema: no se toca.

Qué es: lee los movimientos bancarios. Verifica que el contenido sea una lista, descarta objetos sin `transaccion_id` dejando advertencia, y traduce cada movimiento a `MovimientoBancario` conservando su identificador propio `MOVxxxx`.

Cuando el JSON es inválido, el error incluye **línea y columna** del problema, que es lo que necesita quien tenga que arreglarlo.

9. `limpieza/` — interpretar lo sucio

Es la parte más delicada del proyecto y donde se concentra la dificultad real de la prueba.

`reconciliacion/limpieza/montos.py`

120 líneas

Qué es: extrae el monto del campo `Monto` del CSV, que no es un número sino una estructura rota.

Estrategia: en vez de *reparar* el JSON —frágil, porque hay más de una variante de corrupción— se **extrae con una expresión regular tolerante** que busca la clave `monto` sin importar cómo venga escapada alrededor.

Los tres formatos que conviven en el archivo:

Como viene	Valor real	Trampa
1250000	1.250.000	—
"188.000 COP"	188.000	El punto separa miles
"\$175.000,00"	175.000	Formato colombiano: punto = miles, coma = decimales

El error más caro de la prueba: si se interpreta `$175.000,00` como si el punto fuera decimal, da 17.500.000 y aparecen **53 discrepancias de monto que no existen** (148 en vez de las 95 reales).

Claves duplicadas: `monto` puede aparecer dos veces en la misma fila con valores distintos. Se aplica la semántica estándar de JSON: **gana la última**. Ocurre una sola vez en el archivo (TRX0138) y el valor que gana es el que confirman SQLite y el banco.

`reconciliacion/limpieza/marcas.py`

140 líneas

Qué es: extrae el nombre de la marca —que viaja como último elemento de la lista de retenciones— y lo normaliza para poder compararlo.

El problema: las tres fuentes no escriben la marca igual. El CSV guarda el nombre comercial completo; SQLite y el banco guardan solo su primera palabra.

CSV	SQLite / JSON	Normalizado
NAF NAF	NAF	NAF
AMERICAN EAGLE	AMERICAN	AMERICAN
AMERICANINO	AMERICANINO	AMERICANINO

La regla: mayúsculas, sin tildes ni puntuación, espacios colapsados y **se conserva solo el primer token**.

Comparando literalmente aparecen **183 marcas «distintas»** que en realidad son la misma. Tras normalizar: **cero**. Se eligió el primer token porque el catálogo no tiene dos marcas que compartan la primera palabra —AMERICANINO y AMERICAN EAGLE siguen siendo distintas—, así que no introduce colisiones.

Qué es: extrae las retenciones practicadas, que viven en el mismo campo malformado que la marca.

Las reglas del archivo:

- Entre 1 y 4 retenciones por fila; entidades `iva`, `ica`, `fuentes`, `cree`, `aumento`.
- Una entidad ausente vale **0**.
- Una misma entidad **puede repetirse** con montos distintos: pasa en **267 de las 500 filas**. Hay que sumar todas sus ocurrencias.
- Los valores vienen **negativos** y se conservan con su signo.
- `cree` y `aumento` se parsean para no confundirlas, pero no entran en ninguna de las tres columnas de salida.

La expresión regular empareja entidad y monto **dentro del mismo objeto**, para no aparear la entidad de una retención con el monto de la siguiente.

Las seis variantes de corrupción que conviven en el campo: aperturas `[{`, `{{` o `({`; cierres `}]` o `)}`"; claves con escapes sobrantes; y claves duplicadas y pegadas como `financial_entityfinancial_entity`.

Qué es: convierte las fechas de las tres fuentes, que usan formatos distintos (`26/07/2026 9:00` en CSV y SQLite, `2026-07-26 09:00:00` en JSON).

Prueba varios formatos en orden y, si ninguno encaja, devuelve `None` en vez de lanzar: así la fila se conserva y queda reportada como incidencia, en lugar de perderse.

Qué es: el orquestador de la limpieza. Convierte cada `FilaAutorizacionCruda` en una `Autorizacion` usando los tres extractores.

`ResultadoLimpieza` lleva los contadores que permiten demostrar que se extrajo todo: `sin_monto`, `sin_marca`, `sin_retenciones`, `con_monto_duplicado` y la lista de incidencias.

Criterio de tolerancia a fallos: una fila que no se pueda parsear del todo **no se descarta**. Se conserva con el campo vacío y se registra una incidencia. En un proceso contable es preferible una transacción visible e incompleta que una transacción ausente.

Resultado real: `500/500` autorizaciones limpias (`sin_monto: 0`, `sin_marca: 0`, `sin_retenciones: 0`), con una única incidencia: la clave duplicada de TRX0138.

10. procesamiento/ — decidir

reconciliacion/procesamiento/reglas.py

221 líneas

Qué es: las reglas de clasificación, cada una como una clase con una sola responsabilidad y todas con la misma interfaz (patrón **Strategy**).

Regla	Qué decide
ReglaPresencia	En qué fuentes falta la transacción
ReglaMonto	Si los montos difieren entre fuentes
ReglaEstado	Si la contabilidad no la dio por buena
ReglaFechas	Si las fechas se separan más de 2 horas
ReglaBanco	Si las fuentes reportan bancos distintos
ReglaReconciliado	Marca lo que quedó sin ningún hallazgo

El orden importa: `ReglaReconciliado` va de última porque `RECONCILIADO` es la única etiqueta excluyente y solo aplica si ninguna otra encontró nada.

La regla de estados, que es la trampa del enunciado: cada fuente usa su propio vocabulario (`AUTORIZADO` , `CONTABILIZADO` , `COMPLETADO`) y los tres significan lo mismo. **No** se marca discrepancia porque los textos difieran: solo cuando SQLite dice `PENDIENTE` o `RECHAZADO` .

`ReglaPresencia` **cubre las siete combinaciones posibles**, no solo las tres que nombra el enunciado, para que ninguna transacción del universo quede sin clasificar aunque aparezca un caso que hoy no está en los datos.

Los hallazgos que el enunciado no nombra —desfase de fechas, banco distinto— se dejan como **observación** y no como etiqueta: comprobarlos sí, pero sin inventar vocabulario que nadie acordó.

reconciliacion/procesamiento/reconciliador.py

159 líneas

Qué es: el motor. Cruza las tres fuentes por el identificador y aplica las reglas.

Cómo funciona: construye un diccionario por fuente, calcula la **unión** de los identificadores, crea una `TransaccionReconciliada` por cada uno y le pasa las reglas en orden.

`ResumenReconciliacion` calcula los indicadores agregados —total, reconciliadas, conteo por etiqueta, monto total y monto en discrepancia— que se muestran **en la interfaz y no en el Excel**, como pide el enunciado.

Las reglas se le pueden **inyectar por constructor**, que es lo que usan las pruebas para aislar una sola.

Qué es: la detección de fraude, que es una **dimensión independiente** de la clasificación: una transacción puede estar RECONCILIADO y aun así ser fraude. Ocurre en **90 de las 505**.

Regla	Definición	Detectadas
ReglaMontoAnomalo	Monto de referencia mayor que media + 3σ	11
ReglaHoraInusual	Fecha de referencia entre 00:00 y 05:59	134
ReglaPatronSospechoso	Mismo banco y mismo monto en ≤ 60 minutos	12
ReglaSinAutorizacion	Existe en el banco pero no en el CSV	5

La diferencia técnica con las reglas de clasificación: aquí el contrato recibe **toda la colección** y no una transacción suelta, porque tres de los cuatro patrones son estadísticos o relacionales y no se pueden evaluar de forma aislada.

Desviación estándar poblacional, no muestral: el conjunto analizado no es una muestra de algo mayor, es la población completa del periodo. Sobre estos datos ambas dan el mismo resultado (11 transacciones), pero el criterio queda explícito. El umbral calculado es **\$733.773,55**.

En el patrón sospechoso se marcan **todas** las transacciones involucradas, no solo la segunda de cada par, y la observación dice con cuáles cruza.

11. `exportadores/` — presentar

`reconciliacion/exportadores/excel.py`

309 líneas

Qué es: genera el entregable principal: una hoja `Reconciliacion` con la tabla desde A1, 29 columnas y una fila por transacción.

La decisión de diseño: las 29 columnas se declaran como **datos** (una lista de `ColumnaReporte` con su título, cómo obtener el valor y con qué formato mostrarse), no como 29 bloques de código repetido. El método que escribe la hoja es el mismo sin importar cuántas columnas haya, y mover o agregar una es editar una línea.

Formato aplicado:

- Encabezados en negrita, con fondo azul, fila congelada y autofiltro.
- Montos como **número** y fechas como **datetime** reales con formato de Excel —no como texto—, para que se pueda filtrar, sumar y ordenar sin convertir nada.
- Ancho de columna ajustado al contenido.
- **Color de fila por precedencia:** naranja si hay fraude; si no, rojo ante cualquier hallazgo; si no, verde.

Resultado: 149 filas naranjas, 156 rojas y 200 verdes. Un buen ejemplo es TRX0001: está `RECONCILIADO` pero es `FRAUDE_MONTO`, así que sale naranja. Esa fila sola demuestra que la precedencia funciona y que clasificación y fraude son dimensiones distintas.

Si el archivo está abierto en Excel, lanza `ErrorExportacion` con un mensaje que le dice al usuario que lo cierre.

12. `gui/` — la interfaz

`reconciliacion/gui/mensajes.py`

57 líneas

Qué es: los mensajes que el hilo de trabajo envía a la ventana: `PROGRESO`, `LOG`, `ERROR`, `CANCELADO` y `FIN`.

Usar un tipo explícito en vez de tuplas sueltas hace que el bucle de la interfaz sea legible y que agregar un aviso nuevo no rompa lo que ya existe.

Qué es: el hilo que ejecuta la reconciliación sin congelar la ventana.

Qué publica en la cola: el avance que le informa el servicio, y las líneas de log del negocio, capturadas con un `logging.Handler` propio (`ManejadorCola`). Así la bitácora de la pantalla es exactamente la misma que la de la terminal, sin que el negocio sepa que existe una interfaz.

Cancelación: `cancelar()` no mata el hilo —matarlo dejaría el trabajo a medias— sino que levanta una bandera que el propio hilo revisa en su siguiente aviso de avance.

Lo que costó de verdad: el patrón hilo + cola + `after()` es el correcto, y **aun así la ventana se congelaba 1,57 segundos**. La causa era el GIL: este hilo usa 100 % de CPU y el intérprete solo lo interrumpe cada 5 ms, dejando al hilo de la ventana sin turnos. Se corrigió con una pausa real de 1 ms en cada aviso —`sleep(0)` no sirve en Windows— y bajando el intervalo de conmutación del intérprete a 0,5 ms mientras dura el proceso.
Resultado medido: el bloqueo máximo bajó de 1,57 s a 0,15 s.

Qué es: la ventana principal, organizada en cuatro bloques que siguen el orden mental de quien la usa: fuentes de datos → acción y avance → indicadores → detalle del proceso.

Métodos principales:

Método	Qué hace
<code>_ajustar_al_monitor()</code>	Dimensiona la ventana contra la pantalla disponible
<code>_revisar_fuentes()</code>	Punto verde o rojo por archivo, antes de ejecutar
<code>_ejecutar()</code>	Lanza el hilo de trabajo
<code>_procesarCola()</code>	Vacía la cola y actualiza la ventana; se reprograma solo cada 60 ms
<code>_atender()</code>	Aplica cada tipo de mensaje
<code>_mostrar_resultado()</code>	Vuelca los cuatro indicadores
<code>_guardar_bitacora()</code>	Exporta el detalle a un .txt
<code>_al_cerrar()</code>	Detiene el hilo y cancela el sondeo pendiente

`_procesarCola()` **es el único punto** donde se tocan los widgets con datos producidos por otro hilo, y corre siempre en el hilo de la interfaz. Atiende como máximo 250 mensajes por vuelta y acumula las líneas de bitácora para escribirlas de una sola vez, porque insertarlas una por una es lo que más cuesta del ciclo.

Detalles que solo aparecieron probando: los colores se declaran como pares (tema claro, tema oscuro) porque con un solo tono los botones deshabilitados eran invisibles sobre blanco; y la ventana se dimensiona contra el monitor porque en una pantalla de 1366×768 el mensaje con la ruta del reporte quedaba fuera de la pantalla.

Qué es: expone el paquete, pero **importa la ventana de forma perezosa**. Importarla de entrada arrastraría `tkinter` y exigiría un entorno gráfico; el hilo de trabajo, en cambio, solo habla con una cola y se puede probar en cualquier parte. Es lo que permite que la integración continua corra en un servidor Linux sin pantalla.

13. tests/ — las pruebas

172 pruebas, repartidas donde de verdad se puede romper algo.

Archivo	Líneas	Qué cubre
conftest.py	162	Fábricas de modelos, para no construirlos a mano en cada prueba
test_montos.py	74	Los tres formatos y la clave <code>monto</code> duplicada
test_marcas.py	83	Extracción y la regla del primer token
test_retenciones.py	108	Emparejamiento, entidades repetidas, signos y las tres sumas
test_reglas.py	201	Reglas de clasificación y las siete combinaciones de presencia
test_fraude.py	177	Los cuatro patrones y sus bordes exactos
test_transaccion.py	82	Monto y fecha de referencia, diferencias, presencia
test_loaders.py	227	Carga y errores: archivo ausente, JSON inválido, tabla inexistente
test_excel.py	158	Estructura, formatos y precedencia de color
test_trabajador.py	156	Cancelación, errores y avance, sin abrir ninguna ventana
test_servicio.py	142	La cadena completa sobre los datos reales

Tres decisiones sobre cómo están escritas:

- **Los casos de los parsers salen del archivo real.** No son cadenas inventadas: son las variantes de corrupción que efectivamente aparecen en `autorizaciones.csv`, incluido el caso concreto de TRX0138.
- **Se prueban los bordes exactos:** 05:59 sí es hora inusual y 06:00 no; 60 minutos sí es patrón y 61 no.
- **test_servicio.py fija los números.** Corre el proceso completo y afirma 505 transacciones, 290 reconciliadas, 149 fraudes y el conteo exacto por etiqueta. Si alguien toca un parser o una regla y los totales se mueven, esa prueba lo dice en el acto.

Cobertura: 77 % total; **95 %** sin contar la ventana, que se excluye a propósito porque probar píxeles es caro y frágil. `mypy --strict` pasa sin errores en 32 archivos.

14. Carpetas de datos y salidas

Carpeta	Qué contiene	Se versiona
datos/	Las tres fuentes de entrada	Sí, para que el repositorio se pueda clonar y correr
salida/	<code>reporte_reconciliacion.xlsx</code>	Sí, es un entregable. Las bitácoras exportadas no
capturas/	Las tres imágenes de la interfaz	Sí, se muestran en el README
docs/	Esta documentación y el manual de uso	Sí

15. Patrones de diseño aplicados

Patrón	Dónde	Qué problema resuelve
Template Method	CargadorFuente	Los cuatro pasos de la carga son iguales; solo cambia cómo se lee
Strategy	Reglas de clasificación y de fraude	Agregar o cambiar un criterio no obliga a tocar el motor
Inyección de dependencias	Reconciliador, detector, servicio	Permite sustituir piezas en las pruebas
Configuración como datos	Las 29 columnas del Excel	Evita 29 bloques de código repetido
Productor / consumidor	Hilo de trabajo ↔ interfaz	La ventana nunca se bloquea
Objeto de valor	Modelos del dominio (inmutables)	Un dato cargado no cambia por accidente

16. Reglas de negocio, de referencia

Clasificación (columna Clasificacion)

Etiqueta	Cuándo aplica	Casos
RECONCILIADO	En las 3 fuentes, montos idénticos y estado contable OK. Es excluyente	290
DISCREPANCIA_MONTO	El monto difiere entre las fuentes presentes	95
DISCREPANCIA_ESTADO	SQLite dice PENDIENTE o RECHAZADO	55
NO_ENCONTRADO_EN_JSON	Autorizada pero no llegó al banco	40
NO_CONTABILIZADO	Autorizada pero sin registro contable	20
NO_AUTORIZADO	El banco la reporta pero nadie la autorizó	5

Fraude y nivel de riesgo

Nivel	Cuándo	Casos
CRÍTICO	FRAUDE_NO_AUTORIZADO	5
ALTO	FRAUDE_MONTO	6
MEDIO	FRAUDE_HORA o FRAUDE_PATRON	138

Resultado final sobre los datos entregados

Indicador	Valor
Universo analizado	505 transacciones
Reconciliadas	290 (57,4 %)

Sin clasificar	0
Monto total	\$124.690.000
Monto en discrepancia	\$475.000
Con al menos un patrón de fraude	149
Filas del CSV con extracción completa	500 de 500
Duración del proceso	≈ 1,2 segundos